Thomas Eastwood Individual Work:

The singleton design pattern can be implemented into our existing code quite easily, so I did exactly that. Instead of simply having a Session class, I made a getSession() function as seen above, which returns a new static variable ‘instance’. This instance is the only session allowed to exist. The Session() constructor has become private, and is only called within the getSession() function which handles the logic of creating a session. In the future, we can implement user permissions here and integrate login logic to ensure that a user is only given access to what they should have access to by overloading the getSession function, and using internal logic to differentiate between users and admins. Right now, we are using a developer role that will not exist in the final product, unless needs change to justify its differentiation from the admin role.

```java
//Session object class to keep one connection open, instead of using a new connection to AWS for every function call
public static class Session {

    //Attributes
    private static Session instance;        //instance variable to enforce singleton design pattern
    private final String auth;
    private final String messagesTableName;
    private final String accountsTableName;
    private final DynamoDbClient client;

    //Accessors
    public String getAuth(){return auth;}
    public String getMessagesTableName(){return messagesTableName;}
    public String getAccountsTableName(){return accountsTableName;}
    public DynamoDbClient getClient(){return client;}

    //Close Session
    public void close(){
        client.close();
    }
```

```
//Constructor
private Session(String auth, Region region){

    //Setup String identifiers
    this.auth = auth;
    this.messagesTableName = "parakeetMessages";
    this.accountsTableName = "";

    //Setup region and client
    // Connect to DynamoDB in Oregon (us-west-2)
    this.client = DynamoDbClient.builder()
        .region(region)
        .credentialsProvider(ProfileCredentialsProvider.create(auth))
        .build();
}

public static Session getSession(){

    //Creating a new instance if none exists
    if(instance==null){
        instance = new Session("parakeet-dev", Region.US_WEST_2);
    }

    //Returning instance that may or may not have already existed
    return instance;
}
```